

Alocação de Computação de Inferência na Síntese de Programas com Modelos de Linguagem

Anonymous submission

Resumo

Escalar a computação em tempo de inferência tornou-se o principal eixo de ganho em síntese de programas por modelos de linguagem, e um mesmo orçamento de chamadas admite duas alocações incompatíveis: *diversificar*, gerando N candidatos independentes, ou *iterar*, revisando um único candidato com feedback. A literatura relata ganhos para a segunda via, mas também falhas sistemáticas de autocorreção; as duas leituras são difíceis de separar porque, sem informação externa, o revisor não sabe que errou. Este trabalho remove esse confundimento por construção, concedendo à via iterativa uma vantagem que ela não teria em produção: um Agente Crítico que enxerga o gabarito do par de teste e devolve apenas contradições em linguagem natural, sem propor solução e sem participar da seleção do candidato final. O crítico opera assim como um limite superior otimista da via iterativa. Avaliamos as duas alocações no ARC-AGI-1 em um desenho pareado com orçamento igualado de 7 chamadas por tarefa. Em 270 tarefas, a revisão guiada por oráculo **não** superou a amostragem independente ($-0,7$ pp; McNemar exato $p = 0,8877$), e o intervalo de confiança limita qualquer vantagem do crítico a $+4,3$ pp. Uma decomposição fatorial nas mesmas 270 tarefas, com quatro condições que variam isoladamente o acesso ao oráculo e a forma do feedback, tampouco as separa: **nenhuma** das 5 comparações pré-registradas é significativa — nem sob Bonferroni, nem a $\alpha = 0,05$ sem correção. O achado que consideramos mais transferível independe do sinal do efeito: cerca de metade das vitórias de qualquer condição é decidida na primeira geração, que é idêntica entre elas. Descontada essa parcela, o experimento discrimina em cerca de um quinto da amostra e a ordenação entre as condições inverte em relação ao número bruto — um confundimento herdado por qualquer comparação entre esquemas de inferência que partam do mesmo *prompt*, e que nenhuma métrica agregada revela.

1 Introdução

Durante uma década, os ganhos em modelos de linguagem vieram de escalar parâmetros e dados de treino. O eixo mudou: hoje uma fração substancial do desempenho em tarefas de raciocínio vem de escalar a computação no momento da inferência — gastar mais chamadas, mais tokens e mais

tempo por problema (Snell et al. 2024; Brown et al. 2024). Isso converte uma questão antes implícita em uma decisão de engenharia com consequência direta de custo: dado um orçamento fixo de chamadas a um modelo, como alocá-lo?

Há duas respostas estruturalmente distintas. A primeira é **diversificar**: gerar N programas independentes e selecionar o melhor por um verificador barato. É a receita de *pass@k* em geração de código (Chen et al. 2021), da filtragem massiva do AlphaCode (Li et al. 2022) e da autoconsistência em raciocínio (Wang et al. 2023). A segunda é **iterar**: gastar o mesmo orçamento revisando um único candidato à luz de feedback, como em *self-debugging* (Chen et al. 2024), SELF-REFINE (Madaan et al. 2023) e REFLEXION (Shinn et al. 2023).

A evidência sobre a segunda via é contraditória. Os trabalhos originais relatam ganhos expressivos; réplicas controladas encontram que grande parte do ganho desaparece quando o orçamento é igualado, ou quando o modelo precisa detectar sozinho que errou (Huang et al. 2024; Stechly, Marquez, and Kambhampati 2023; Olausson et al. 2024). A explicação candidata mais simples é que autocrítica não é crítica: um modelo sem acesso a informação externa sobre o próprio erro tende a reafirmar a resposta que já produziu. Sob essa hipótese, o fator limitante não é a estratégia iterativa em si, mas a qualidade do canal de feedback.

Este trabalho testa essa hipótese maximizando exatamente a variável que ela aponta. Nosso Agente Crítico enxerga o gabarito do par de teste, que o Gerador nunca vê, e sua função é restrita por construção: não propõe solução, não escreve código e não reproduz grades, limitando-se a apontar em que pontos a regra enunciada e o programa candidato contradizem a realidade. Um filtro automático redige qualquer grade ou trecho de código que escape em sua saída, e o gabarito nunca participa do critério de parada nem da seleção do candidato final.

O desenho é, portanto, deliberadamente favorável à via iterativa: o crítico dispõe de informação privilegiada que nenhum sistema autônomo teria e ainda assim consome o mesmo orçamento de chamadas que a amostragem. Ele funciona como um **limite superior otimista**, o que torna a leitura do resultado informativa nos dois sentidos. Se o crítico vencer, o gargalo da autocorreção é informacional, e investir em verificadores externos se justifica. Se não vencer — mesmo com o gabarito disponível —, a vantagem atribuída ao feedback

iterativo fica sob suspeita muito além deste arranjo específico.

Avaliamos no ARC-AGI-1 (Chollet 2019), um conjunto de tarefas de raciocínio abstrato em que cada item traz alguns pares entrada → saída de demonstração e exige inferir a transformação subjacente. O domínio é adequado à questão por três razões. A resposta é verificável exatamente — a grade de saída está inteiramente correta ou incorreta —, o que elimina a ambiguidade de métricas parciais. Os pares de demonstração fornecem um verificador barato e honesto que ambas as estratégias podem usar sem tocar no gabarito. E o *benchmark* permanece resistente a modelos de propósito geral (Chollet et al. 2024; Gendron et al. 2024), deixando espaço de medição que *benchmarks* saturados não deixam.

Contribuições.

- Um protocolo pareado com orçamento de chamadas igualado que isola a decisão diversificar vs. iterar de confundimentos de recursos, com salvaguardas explícitas e auditáveis contra contaminação pelo oráculo (Seções 3 e 3.4).
- Um resultado nulo com limite superior estreito em 270 tarefas: não apenas a ausência de efeito detectado, mas a delimitação de qualquer efeito a menos de 4,3 pp (Seção 4.2).
- Uma decomposição fatorial do mecanismo em dois eixos — acesso ao oráculo e forma do feedback — executada nas mesmas 270 tarefas, com uma família de 5 comparações pré-registradas sob correção de Bonferroni (Seção 4.1).
- O *desconto da primeira geração*: a quantificação de quanto de uma comparação entre arquiteturas de agentes é atribuível à arquitetura. Metade das vitórias de qualquer condição não pertence a nenhuma delas, o experimento discrimina em cerca de um quinto da amostra, e a ordenação entre as condições inverte quando se desconta essa parcela (Seção 4.2).
- A documentação empírica de instabilidade em três escalas de amostra e em casos individuais, incluindo uma extrapolação de análise interina que projetou $p = 0,0027$ para um resultado que fechou em $p = 0,0789$ (Seção 4.3).

Em síntese, sob orçamento igualado a revisão guiada por oráculo **não** superou a amostragem independente, e **nenhuma** das 5 comparações pré-registradas é significativa — nem sob Bonferroni, nem a $\alpha = 0,05$ sem correção. O crítico cumpre o que promete no nível do mecanismo — atinge a mesma consistência com os pares de demonstração usando 40% menos programas — sem que isso se converta em acerto no par de teste. E cerca de metade das vitórias de qualquer condição é decidida na primeira geração, que é idêntica entre elas.

2 Trabalho Relacionado

Alocação de orçamento em largura. Amostrar muitos candidatos e filtrar é a estratégia de referência em síntese de programas com modelos de linguagem desde o Codex (Chen et al. 2021). Li et al. (2022) levaram a ideia ao limite,

amostrando ordens de magnitude mais candidatos do que seriam submetidos e filtrando por execução em testes. Wang et al. (2023) demonstraram o análogo em raciocínio, agregando cadeias amostradas por votação. Trabalhos recentes caracterizam a curva: a cobertura cresce de forma previsível com o número de amostras (Brown et al. 2024), e há regimes em que alocar computação de inferência supera aumentar o modelo (Snell et al. 2024). O elo fraco dessa família é o *seletor* — como escolher entre os N candidatos sem acesso à resposta —, endereçado na literatura por verificadores aprendidos sobre a resposta final (Cobbe et al. 2021) ou sobre passos intermediários (Lightman et al. 2024). Nossa condição *sampling* é uma instância deliberadamente simples dessa família, com o verificador mais honesto que o ARC oferece sem custo adicional — os próprios pares de demonstração —, o que mantém a comparação livre de qualquer componente treinado.

Alocação de orçamento em profundidade. A via iterativa realimenta o modelo com o resultado da execução ou com uma crítica textual e solicita uma revisão. Chen et al. (2024) usam a saída do interpretador como feedback; Madaan et al. (2023) e Shinn et al. (2023) usam crítica gerada pelo próprio modelo. A contestação foi rápida: Huang et al. (2024) argumentam que modelos não corrigem raciocínio sem sinal externo, Stechly, Marquez, and Kambhampati (2023) mostram que a melhora aparente provém do verificador embutido no protocolo e não da autocrítica, e Olausson et al. (2024) constatam que o auto-reparo em código só compensa quando o crítico é mais capaz que o gerador. Nosso desenho toma a variável que essa literatura identifica como determinante — a qualidade do sinal externo — e a maximiza.

Contraexemplos como canal de feedback. Guiar síntese por contraexemplos antecede os modelos de linguagem: em CEGIS (*Counterexample-Guided Inductive Synthesis*), um verificador devolve ao sintetizador uma entrada concreta em que o candidato falha, e o laço converge sobre uma especificação (Solar-Lezama et al. 2006; Alur et al. 2013; Gulwani, Polozov, and Singh 2017). Nossa condição *critic_cegis* transpõe esse protocolo para o registro de linguagem natural, restringindo a crítica a um contraexemplo e a uma classe de correção de vocabulário fechado, o que permite variar a forma do feedback mantendo o conteúdo informacional constante.

ARC-AGI e síntese de programas. O ARC foi proposto como medida de eficiência de aquisição de habilidade (Chollet 2019) e resiste a modelos gerais (Gendron et al. 2024; Mirchandani et al. 2023). Abordagens competitivas o tratam como síntese de programas guiada por busca sobre hipóteses em linguagem natural (Wang et al. 2024) ou por amostragem massiva de programas (Greenblatt 2024), na linhagem de sistemas de síntese indutiva como o DreamCoder (Ellis et al. 2021). Não propomos um novo estado da arte no *benchmark*: reportamos uma medição controlada de alocação de orçamento dentro de um arcabouço de síntese fixo.

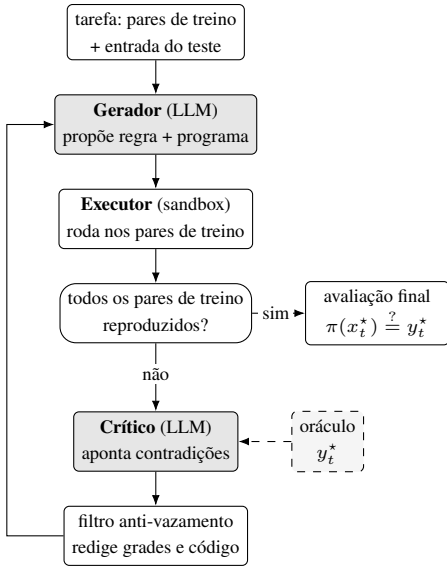


Figura 1: Arquitetura do solucionador. O Gerador nunca vê y_t^* em nenhuma condição; o Crítico o vê (linha tracejada) apenas nas condições que o concedem. A restrição central do desenho é que y_t^* não decide nada: o critério de parada e a seleção do candidato final consideram apenas os pares de demonstração, e a avaliação final é executada uma única vez, ao término do laço, exclusivamente para medir.

3 Metodologia

3.1 Formulação do Problema

Uma tarefa t do ARC-AGI-1 é um par $(\mathcal{D}_t, (x_t^*, y_t^*))$, em que $\mathcal{D}_t = \{(x_i, y_i)\}_{i=1}^k$ são os pares de demonstração e (x_t^*, y_t^*) é o par de teste. Grades são matrizes de inteiros em $\{0, \dots, 9\}$ com dimensões variáveis.

Um *solucionador* é um procedimento que, recebendo $\mathcal{D}_t$ e x_t^* , produz um programa π em Python. A tarefa é considerada resolvida se e somente se $\pi(x_t^*) = y_t^*$ com igualdade exata da grade inteira. A métrica reportada é a acurácia $\text{acc} = \frac{1}{|T|} \sum_{t \in T} \mathbf{1}[\pi_t(x_t^*) = y_t^*]$.

O recurso controlado é o **orçamento de chamadas B** : o número máximo de requisições ao modelo de linguagem que o solucionador pode emitir por tarefa, somando todos os papéis de agente. Comparamos duas políticas $\mathcal{S}$ (diversificar) e $\mathcal{C}$ (iterar) de gasto de B , e a hipótese sob teste é

$$\Delta = \mathbb{E}_t[\mathbf{1}[\mathcal{C} \text{ resolve } t]] - \mathbb{E}_t[\mathbf{1}[\mathcal{S} \text{ resolve } t]] > 0, \quad (1)$$

com B mantido idêntico nas duas políticas. Essa igualdade é o controle experimental central da Equação 1: sem ela, a condição com mais recursos venceria por dispor de mais recursos, e a diferença observada não seria atribuível à estratégia.

3.2 Arquitetura do Solucionador

A Figura 1 descreve o laço comum a todas as condições. O Gerador recebe os pares de demonstração e a entrada de teste e propõe uma regra em linguagem natural acompanhada

de um programa. O Executor roda o programa em *sandbox* sobre os pares de demonstração. Se todos forem reproduzidos, o laço termina; caso contrário, e havendo orçamento, o Crítico examina o candidato e devolve um feedback que passa pelo filtro anti-vazamento antes de retornar ao Gerador. As condições diferem apenas em como consomem o orçamento dentro desse laço e no que o Crítico enxerga.

3.3 Condições Experimentais

sampling (diversificar). Aloca o orçamento em largura. Cada uma das B chamadas é uma conversa nova com o mesmo *prompt* inicial: o modelo não sabe que já tentou antes, não vê o resultado da execução e não recebe crítica. Ao final, vence o candidato que reproduz mais pares de demonstração.

critic (iterar sob oráculo). Aloca o orçamento em profundidade. O Gerador propõe regra e programa; o Crítico, que enxerga y_t^* , aponta contradições em prosa livre; o Gerador revisa com o histórico completo da conversa. Cada ciclo custa duas chamadas, de modo que com $B = 7$ resultam 3,23 programas contra os 5,43 da amostragem. A hipótese é que a densidade informacional do feedback compense a redução no número de tentativas.

A adoção de *best-of- N* como linha de base é deliberada. Nada obriga quem dispõe de B chamadas a gastá-las revisando um único programa em série; medir a intervenção contra uma alternativa que aloca o orçamento pior produziria uma vantagem que informa mais sobre o comparativo escolhido do que sobre o método.

Decomposição do mecanismo. A comparação entre essas duas condições responde a uma questão pragmática, mas não a uma causal: elas diferem em quatro eixos simultâneos — existência de feedback, acesso ao gabarito, orçamento gasto em paralelo ou em série, e temperatura. Para separar as duas dimensões de maior interesse, definimos duas condições adicionais, cada uma variando um único eixo em relação a *critic* (Figura 2):

- **critic_no_oracle** — recebe os pares de demonstração, a regra, o código e o relatório de execução, mas nunca y_t^* . Isola quanto do efeito de uma crítica estruturada decorre apenas da estrutura, sem informação privilegiada.
- **critic_cegis** — recebe o mesmo que *critic*, inclusive y_t^* , mas responde com um único contraexemplo e uma classe de correção em vocabulário fechado (MISSING_CASE, WRONG_TRANSFORM, WRONG_GEOMETRY, WRONG_COLOR_MAP, WRONG_SCOPE, OTHER), em vez de prosa livre. Isola o efeito da forma do feedback com o acesso à informação mantido constante.

As duas são condições paralelas e independentes, não um *pipeline* sequencial. Um *pipeline* (Gerador → crítico sem oráculo → crítico de contraexemplo) foi considerado e descartado por variar as duas dimensões simultaneamente, o que impediria atribuir o efeito a um mecanismo específico. A Tabela 1 consolida a definição operacional das quatro condições.

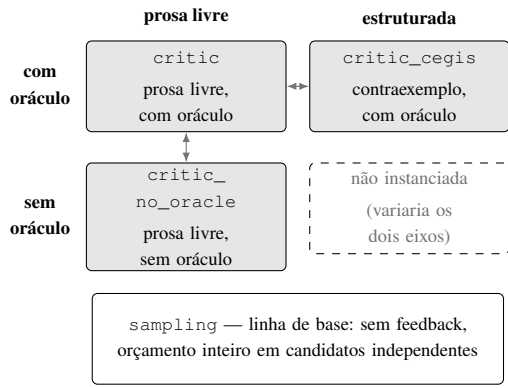


Figura 2: As quatro condições dispostas nos dois eixos que a decomposição pretende separar. As setas marcam as comparações que variam um único eixo: na horizontal, a forma do feedback com o acesso à informação constante; na vertical, o acesso à informação com a forma constante. A célula inferior direita não é instanciada porque variaria os dois eixos ao mesmo tempo.

	samp	critic	no_or	cegis
Feedback	—	prosa	prosa	estrut.
Vê y_t^*	não	sim	não	sim
Vê o código	—	sim	sim	sim
Histórico (Gerador)	não	sim	sim	sim
Memória (Crítico)	—	não	não	não
Temperatura	0,8	0,2	0,2	0,2
Chamadas por ciclo	1	2	2	2
Programas/tarefa	7	4	4	4
Desempate	1 ^a	últ.	últ.	últ.

Tabela 1: Definição operacional das quatro condições, abreviadas nos cabeçalhos (no_or = critic_no_oracle, cegis = critic_cegis). Todas competem sob o mesmo orçamento B de chamadas; variam a alocação desse orçamento e o que o Crítico enxerga. “Desempate” indica qual candidato vence quando há empate na consistência com os pares de demonstração.

3.4 Salvaguardas Contra Contaminação pelo Oráculo

Conceder o gabarito a um agente cria três riscos distintos, tratados por mecanismos distintos.

O gabarito não seleciona. O critério de parada é idêntico nas quatro condições: o candidato é aceito quando reproduz todos os pares de demonstração, e o candidato final é o melhor por esse mesmo critério. O par de teste é executado uma única vez, ao final, apenas para medir. Se o laço parasse quando o código acertasse o teste, a taxa de acerto passaria a ser um limite superior de oráculo, não uma medida de desempenho. Essa é a distinção entre usar o gabarito para formular feedback e usá-lo para escolher a resposta; apenas a primeira está sob teste.

O gabarito não vaza. O feedback do Crítico passa por um filtro automático antes de chegar ao Gerador: são redigidas

linhas compostas apenas de inteiros (grades, listas de coordenadas) e qualquer código, seja em blocos cercados ou em linhas contendo `def`, `import`, `return`, `for` ou `while`. A resposta bruta é preservada nos registros para auditoria e cada redação é contabilizada. O vazamento deixa assim de depender da cooperação do modelo e torna-se um evento mensurável: uma contagem alta coloca a condição sob suspeita e é reportada.

O Crítico não resolve. O Crítico é reinstituído a cada rodada, sem histórico das anteriores. Sem memória, não pode construir uma solução ao longo do diálogo, e cada crítica é um julgamento independente do candidato apresentado. Um crítico capaz de acumular raciocínio seria um segundo solucionador, e a comparação deixaria de ser sobre alocação de orçamento.

3.5 Parâmetros de Configuração

Temperatura assimétrica. `sampling` opera a $T = 0,8$ e as condições de crítico a $T = 0,2$. Amostragem sem diversidade não é amostragem: a $T = 0,2$ as N amostras colapsam em programas quase idênticos e o *best-of- N* degenera em uma tentativa repetida N vezes. O custo metodológico é que a temperatura passa a ser mais um eixo de diferença entre as condições. Ela é constitutiva da estratégia e não uma variável de perturbação, mas parte de uma eventual vantagem de `sampling` decorre de explorar um espaço maior — o que é a estratégia operando, não um artefato.

Regra de desempate. Empates na consistência com os pares de demonstração são resolvidos pela última iteração nas condições de crítico e pela primeira amostra em `sampling`. Em `critic`, a iteração mais recente incorporou feedback, e preferi-la é coerente com a estratégia; em `sampling` não há progressão, e preferir a última faria com que amostrar mais deslocasse a escolha por si só, conferindo à estratégia uma vantagem independente da qualidade dos candidatos.

Paridade do orçamento. Uma condição de crítico só invoca o Crítico se ainda couberem duas chamadas, para não encerrar o orçamento em um feedback inutilizável. A última chamada de uma tarefa é, portanto, sempre do Gerador, e B deve ser ímpar: com $B = 6$, a condição usaria 5 chamadas enquanto `sampling` usaria 6.

Modelo único nos dois papéis. Gerador e Crítico usam o mesmo modelo. Se o Crítico fosse um modelo distinto, uma eventual vantagem poderia ser atribuída à capacidade do segundo modelo em vez da assimetria de informação, que é o que está sob teste.

3.6 Seleção do Modelo

Um desenho pareado só tem poder estatístico se existirem tarefas que uma estratégia resolve e a outra não. Modelos em qualquer dos extremos destroem essa propriedade de formas espelhadas: um modelo forte resolve a tarefa na primeira geração — idêntica nas duas condições — e produz apenas pares concordantes; um modelo fraco falha em ambas e também produz concordância. A Tabela 2 reporta a medição que orientou a escolha.

Modelo	Acur. (%)	Ambas falham	Discord.
3.7-flash	~90	~0%	~0
3.5-flash-lite	26/28	66%	7 (14%)
3.1-flash-lite	13,5/16,2	81%	3 (8%)

Tabela 2: Seleção de modelo (prefixo *gemin*i- omitido). Quando duas acurácias são reportadas, correspondem às duas condições comparadas. A coluna decisiva é a última: pares discordantes são as únicas observações que informam qual estratégia é melhor. *gemin*i-3.5-flash-lite é o ponto mais informativo entre os modelos acessíveis, e foi escolhido por essa razão.

A seleção produziu ainda uma restrição de método que registramos por ser transferível: modelos com raciocínio interno não controlável são inadequados a este desenho. Um candidato testado passava, a partir da segunda iteração da condição de crítico, a consumir todo o teto de tokens de saída em raciocínio interno e a devolver texto vazio. O gatilho era especificamente a mensagem de revisão — exatamente o que distingue a condição sob teste. Um procedimento de aprovação que exercitasse apenas o *prompt* inicial teria aprovado o modelo; avaliar um modelo para este desenho exige exercitar as duas formas de *prompt*.

3.7 Protocolo de Análise Estatística

Desenho pareado. As mesmas tarefas rodam em todas as condições, sob o mesmo orçamento. A amostra é sorteada de *evaluation* com semente 20260814, declarada antes da execução, e a lista literal de identificadores é gravada no manifesto da rodada — é ela, e não o algoritmo de amostragem, que sustenta a reprodutibilidade.

Teste de McNemar exato. Em desenhos pareados, o que importa não é a diferença de acurácia agregada, mas quantas tarefas cada condição resolveu que a outra não (McNemar 1947). O teste descarta as tarefas em que as condições concordam e avalia se as discordâncias se dividem por acaso:

$$p = 2P(X \leq \min(a, b)), \quad X \sim \text{Binomial}(a + b, \frac{1}{2}), \quad (2)$$

com a e b as contagens de discordância a favor de cada condição. Adotamos a versão exata em vez da aproximação qui-quadrado, pouco confiável com poucas discordâncias.

Intervalo de confiança de Wilson. Calculado sobre a proporção de discordâncias a favor da intervenção (Wilson 1927):

$$\text{IC}_{95} = \frac{\hat{p} + \frac{z^2}{2n} \pm z \sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}, \quad (3)$$

preferido ao intervalo de Wald, que subestima o erro e pode produzir limites fora de $[0, 1]$. O intervalo é então convertido para pontos percentuais de acurácia. Reportá-lo é o que distingue a ausência de efeito detectado da ausência de capacidade de medição: um resultado nulo com limite superior estreito delimita quanto o efeito não pode ser.

#	Par	Eixo isolado
1	samp vs. critic	hipótese principal
2	samp vs. no_or	crítica estruturada sem oráculo
3	samp vs. cegis	oráculo em forma de contraexemplo
4	critic vs. no_or	acesso ao gabarito, forma constante
5	critic vs. cegis	forma do feedback, acesso constante

Tabela 3: Família de 5 comparações pré-registrada antes da execução do Experimento 2. O par *no_or* vs. *cegis* fica deliberadamente fora por variar os dois eixos simultaneamente.

Duas correções para dois problemas distintos. O estudo faz dois usos de múltiplos testes, cada um exigindo a correção do tipo apropriado.

Múltiplas análises ao longo do tempo. O Experimento 1 teve uma análise interina em 100 tarefas antes da análise principal em 270. Analisar os mesmos dados duas vezes oferece duas oportunidades de cruzar $\alpha = 0,05$: uma simulação sob H_0 com 20.000 réplicas mediu a taxa de erro tipo I subindo de 3,8% (apenas as 270 tarefas) para 6,3% (aceitando qualquer das duas análises). Por essa razão a análise principal foi fixada, antes da extensão da amostra, como sendo a de 270 tarefas; caso ambas fossem reivindicadas, aplicar-se-ia a correção de Pocock (Pocock 1977), com $\alpha = 0,0294$ em cada.

Múltiplas comparações simultâneas. O Experimento 2 pré-registrou uma família de 5 comparações sobre o mesmo conjunto de dados (Tabela 3), o que exige a correção de Bonferroni (Dunn 1961): $\alpha = 0,05/5 = 0,01$ por teste.

Exclusões declaradas. Uma tarefa interrompida por erro de API após as tentativas de reenvio é registrada e excluída da comparação pareada: um modelo indisponível não é evidência sobre o método, e contabilizá-la como falha concederia à outra condição um par discordante que ela não obteve.

3.8 Procedimento Experimental

Todas as condições são avaliadas nas mesmas 270 tarefas sorteadas de *evaluation* com semente 20260814 e $B = 7$, sob o protocolo de análise da Seção 3.7. A medição principal executa as quatro condições da Seção 3.3 sobre esse conjunto (02-03/09/2026, *commit* 14ed956). Uma medição anterior, restrita a *sampling* e *critic* e realizada sobre as mesmas tarefas (21/08/2026, *commit* 5fdda8d), testa a hipótese original isoladamente e é usada como réplica independente na Seção 4.2.

As duas medições diferem em um ponto que precisa ser declarado: a mensagem enviada ao Crítico. Na versão **P1**, usada na medição anterior, ela continha a regra em linguagem natural e o relatório de execução nos pares de demonstração, mas não o código candidato — o Crítico julgava uma descrição do algoritmo, e não o algoritmo. Na versão **P2**, usada na medição principal, o código candidato é incluído. A in-

Condição	Res.	Acur.	Treino	Prog.	Tok.
critic	93/270	34,4%	35,6%	3,23	48,1 k
sampling	88/270	32,6%	35,6%	5,43	28,0 k
cegis	83/270	30,7%	34,8%	3,34	48,8 k
no_oracle	80/270	29,6%	33,3%	3,27	45,8 k

Tabela 4: Resultados agregados nas quatro condições, ordenadas por acurácia. “Treino” é a fração de tarefas em que o candidato final reproduz todos os pares de demonstração — o alvo que as estratégias de fato otimizam; “Prog.” é o número médio de programas gerados por tarefa e “Tok.”, o consumo médio de tokens.

Par	Discord.	p	IC 95%
samp-critic	51 (23×28)	0,5758	−3,3 a +6,7
samp-no_or	54 (31×23)	0,3409	−7,9 a +2,3
samp-cegis	47 (26×21)	0,5601	−6,5 a +3,0
critic-no_or	47 (30×17)	0,0789	−9,1 a +0,2
critic-cegis	54 (32×22)	0,2203	−8,5 a +1,6

Tabela 5: As 5 comparações pré-registradas, com p de McNemar exato e intervalos em pontos percentuais. Nenhuma é significativa sob Bonferroni ($\alpha = 0,01$), e nenhuma cruza sequer $\alpha = 0,05$ isoladamente: o menor p observado é 0,0789.

clusão não abre vazamento: o filtro atua sobre a saída do Crítico, nunca sobre a entrada, e o código é a própria saída do Gerador. (Apêndice A.2)

A condição *sampling* não depende do Crítico e é comum às duas medições; ela foi coletada em 21/08/2026, enquanto as três condições de crítico da medição principal foram coletadas em 02–03/09/2026. As comparações que envolvem a linha de base carregam, portanto, um confundimento temporal, delimitado na Seção 5.3.

4 Resultados

4.1 Decomposição Fatorial em Escala

Com as quatro condições medidas nas mesmas 270 tarefas (Tabela 4), **nenhuma** das 5 comparações pré-registradas é significativa — nem sob Bonferroni, nem a $\alpha = 0,05$ sem correção (Tabela 5, Figura 3). A comparação mais próxima é *critic* vs. *critic_no_oracle*, com $p = 0,0789$, e o intervalo correspondente (−9,1 a +0,2 pp) ainda inclui o zero. A diferença entre *critic* e *sampling* é de +1,9 pp, e o intervalo admite desde −3,3 até +6,7 pp.

O mecanismo, no entanto, opera exatamente como projetado. O Crítico produz 3,23 programas por tarefa contra 5,43 da amostragem — 40% a menos — atingindo a *mesma* consistência com os pares de demonstração (35,6% nas duas condições). Converte orçamento em revisão com eficiência real; o que não ocorre é a conversão disso em acurácia no teste.

A restrição de formato de *critic_cegis* foi obedecida sem exceção: 630/630 das críticas emitidas trouxeram um rótulo válido de classe de correção, e o risco de o modelo escapar do vocabulário fechado não se concretizou em escala.

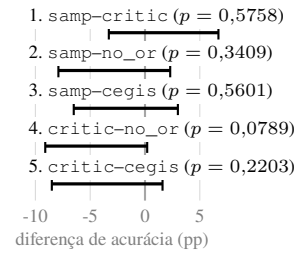


Figura 3: Intervalos de confiança de 95% das 5 comparações pré-registradas. A linha vertical marca a ausência de efeito, e todos os intervalos a cruzam. O que a figura evidencia não é uma ordenação entre condições, e sim a largura dos intervalos: mesmo em 270 tarefas, a medição não separa as hipóteses.

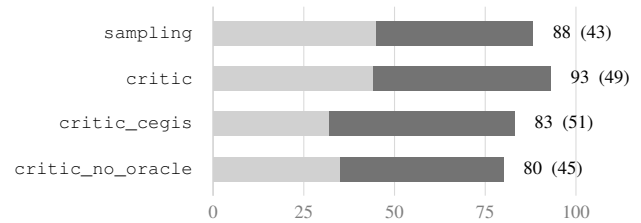


Figura 4: Decomposição das vitórias, em tarefas resolvidas de 270. O segmento claro são as vitórias decididas na primeira geração, comum às quatro condições; o escuro, as atribuíveis ao ciclo. Os rótulos dão o total e, entre parênteses, a parcela do ciclo. A ordenação pelo total (93, 88, 83, 80) *inverte* quando se olha apenas o ciclo (51, 49, 45, 43): as três condições de crítico passam à frente da linha de base.

A adesão ao formato não se traduziu em vantagem. Entre as quatro, *critic_no_oracle* é a de menor acurácia bruta (29,6%) e, ao mesmo tempo, a única que descreve um sistema executável sem gabarito: nesta escala, crítica estruturada sem informação privilegiada não compra nada mensurável. O filtro anti-vazamento registrou 0 eventos nas 1.844 críticas emitidas.

4.2 A Origem das Vitórias

A primeira chamada é idêntica nas quatro condições — mesmo *prompt*, histórico vazio, nenhum crítico envolvido ainda —, diferindo apenas na temperatura. Vitórias decididas ali não distinguem estratégia alguma. A Figura 4 separa, em cada condição, as vitórias obtidas na primeira geração das obtidas pelo ciclo.

O resultado dominante é que cerca de metade das vitórias de qualquer condição é decidida na primeira geração, que é idêntica entre elas. Restringindo cada par às tarefas que a primeira geração não resolveu em nenhuma das duas condições (Tabela 6), a medição discrimina de fato em cerca de um quinto da própria amostra, e todos os 5 p -valores ficam entre 0,2962 e 1,0000.

Duas leituras decorrem daí. A primeira é metodológica: o poder estatístico efetivo de uma comparação entre arquiteturas de agentes é muito menor do que n sugere, porque

Par	<i>n</i>	Líquido	<i>p</i>
samp-critic	207	+7	0,3105
samp-no_or	212	+1	1,0000
samp-cegis	216	+3	0,7201
critic-no_or	214	-7	0,2962
critic-cegis	216	-4	0,6177

Tabela 6: As mesmas 5 comparações restritas às tarefas que a primeira geração não resolveu em nenhuma das duas condições do par. Análise *não* pré-registrada, reportada como exploratória. Todos os *p* ficam entre 0,2962 e 1,0000.

boa parte da amostra é decidida antes de qualquer estratégia agir, ou por nenhuma delas. A segunda é substantiva e vai na direção oposta ao número bruto: descontada a primeira geração, as três condições de crítico ficam *acima* de *sampling* em vitórias do ciclo (51, 49 e 45 contra 43). As diferenças são pequenas e nenhuma é significativa, mas a direção depende de qual quantidade se observa — e essa dependência é invisível em qualquer relato que reporte apenas acurácia agregada.

O efeito replica na medição anterior, com duas condições e sob o protocolo P1: das 88 vitórias de *sampling*, 44 vêm da primeira geração, e das 86 de *critic*, 42 — o que deixa o placar do ciclo em 44×44 , exatamente empatado. Restrito às 206 tarefas que a primeira geração não resolveu, o placar é 18×15 ($p = 0,7283$). Essa medição também não distingue as duas condições em acurácia agregada ($-0,7$ pp, $p = 0,8877$, intervalo de $-5,6$ a $+4,3$ pp): sob orçamento igualado, a revisão guiada por oráculo **não** superou a amostragem independente.

O confundimento é eliminável por construção. Basta que a primeira condição a alcançar uma tarefa grave a primeira geração e as demais a repitam, com a repetição ainda debitada do orçamento — um *cache* de semente. Além de remover o confundimento, ele economiza (condições -1) $\times$ tarefas chamadas por bloco, 810 em um bloco de quatro condições. Nenhuma das medições aqui reportadas o utiliza. (Apêndice A.3)

4.3 Estabilidade das Estimativas

As comparações acima não são significativas, mas a ordenação que elas produzem é frequentemente o que sobra de um estudo desses. Para medir quanto dessa ordenação é reproduzível, subamostramos o conjunto fechado de 270 tarefas sem reposição, preservando o pareamento — as quatro condições avaliadas no mesmo subconjunto — em 20.000 repetições por tamanho.

O resultado está na Figura 5. Com 30 tarefas, a direção observada em 270 — *critic* acima de *sampling* — se reproduz em 51% das subamostras, o que é indistinguível de um sorteio, e a banda de 5 a 95% da diferença de acurácia vai de $-10,0$ a $+13,3$ pp. Com 60 tarefas, ainda 58% e $-6,7$ a $+10,0$ pp. A ordenação completa das quatro condições é substancialmente mais frágil: reproduz-se em 8% das subamostras de 30 tarefas e em 12% das de 60.

Duas consequências. A primeira é que estudos desta classe conduzidos em algumas dezenas de tarefas não estão

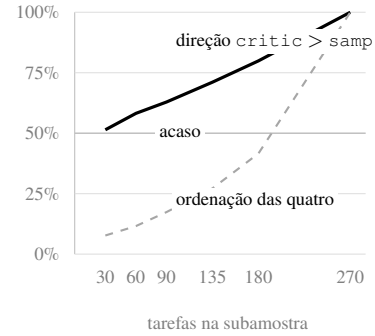


Figura 5: Reprodutibilidade da ordenação sob subamostragem das próprias 270 tarefas (20.000 repetições por tamanho). A linha cheia é a fração de subamostras em que *critic* supera *sampling*; a tracejada, a fração em que a ordenação completa das quatro condições coincide com a observada em 270. Em 30 tarefas a direção é indistinguível de um sorteio (51%) e a ordenação completa se reproduz em 8% das vezes. Ambas as curvas convergem a 100% em 270 por construção.

medindo direção — estão reportando ruído com um sinal, e a ordenação entre mais de duas condições é essencialmente incidental. A segunda é que a subamostragem mede apenas a variância de *amostra*: uma reexecução real acrescenta a variância do próprio modelo, de modo que os valores acima são um piso otimista para a instabilidade observável na prática.

Uma segunda evidência, do mesmo conjunto, quantifica o risco de extrapolar antes do fechamento. Sobre as primeiras 91 tarefas, a comparação *critic* vs. *critic_no_oracle* estava em $p = 0,1153$; extrapolar linearmente aquelas proporções para 270 produzia $p = 0,0027$, valor que teria atravessado a correção de Bonferroni. O conjunto fechado deu $p = 0,0789$, e a vantagem de *critic* sobre *sampling* encolheu de $+6,6$ pp para $+1,9$ pp. A extrapolação errou a significância por mais de uma ordem de magnitude.

É esse o mecanismo que produz literaturas contraditórias sem exigir má conduta: um estudo que parasse em qualquer ponto anterior ao fechamento reportaria uma direção diferente, todos com *p* não significativo, todos reportáveis como observação direcional, e todos ruído (Button et al. 2013; Gelman and Loken 2014; Ioannidis 2005).

4.4 Evidência Qualitativa do Mecanismo

Uma tarefa mostra o que a crítica estruturada faz quando funciona. Ela requer conectar, por segmentos de reta, pares de pixels de mesma cor que compartilham linha ou coluna; o detalhe que invalida soluções ingênuas é que, no cruzamento de linhas de cores diferentes, a vertical tem precedência sobre a horizontal.

sampling falha nas 7 tentativas independentes: sem informação sobre o erro, cada tentativa reinventa uma variação da mesma regra incompleta e nenhuma chega a mencionar o comportamento nos cruzamentos. *critic_no_oracle* resolve em duas rodadas *sem jamais ver o gabarito*: sua

crítica compara o código com a regra que o próprio Gerador enunciou e aponta que a implementação sobrescreve as interseções na ordem em que os segmentos são desenhados, sem precedência alguma — suficiente para localizar o defeito. (Apêndice A.2) É o mecanismo operando no regime para o qual foi projetado, e sem informação privilegiada — o que delimita o quanto do efeito depende do oráculo.

O caso é oferecido como ilustração do mecanismo, não como evidência de sua eficácia agregada. O desfecho de tarefas individuais varia entre execuções da mesma configuração, pela mesma razão que a ordenação varia entre subamostras (Seção 4.3); as conclusões deste trabalho se apoiam no teste pareado sobre a amostra inteira.

5 Discussão

5.1 Interpretação

A leitura defensável é pragmática: sob orçamento de chamadas igualado e neste arranjo, alocar o orçamento em diversidade não é distinguível de alocá-lo em revisão dirigida — nem mesmo quando a revisão é guiada por um oráculo indisponível a qualquer sistema autônomo. Como o desenho favorece a via iterativa por construção, o resultado é mais informativo do que um empate entre esquemas equivalentes: indica que a informação privilegiada, isoladamente, não é o fator limitante.

O achado que consideramos mais consequente, porém, não é sobre qual estratégia vence, e sim sobre *quanto de uma comparação entre arquiteturas de agentes é atribuível à arquitetura*. Cerca de metade das vitórias de qualquer condição é decidida por uma chamada que todas compartilham, e o experimento só discrimina em cerca de um quinto da amostra (Seção 4.2). Esse confundimento não é específico do nosso arranjo: qualquer comparação entre esquemas de inferência que partam do mesmo *prompt* inicial o herda, e ele não aparece em nenhuma métrica agregada.

Os resultados não sustentam a leitura causal de que a assimetria de informação seja irrelevante. As condições diferem em quatro eixos simultâneos, e ainda que a decomposição fatorial isole dois deles, ela o faz sem poder para detectar efeitos da magnitude que os intervalos de confiança admitem.

5.2 Implicações Práticas

A unidade do orçamento altera a conclusão. Contabilizamos o orçamento em chamadas. Contado em tokens, o quadro se deteriora para as condições de crítico: elas consomem de 63% a 74% mais tokens por tarefa que a amostragem (45,8–48,8 k contra 28,0 k), porque o histórico acumulado e o gabarito completo trafegam a cada crítica. Normalizado por *tarefa resolvida* em vez de por tarefa tentada, o Experimento 1 mede a mesma penalidade: 146 k tokens por acerto do Crítico contra 86 k da amostragem, 66% a mais. Sob orçamento medido em tokens — a unidade de custo efetiva — o sinal poderia inverter. Reportamos em chamadas por ser a unidade que torna o número de tentativas comparável, mas a decisão prática deve considerar ambas.

O regime em que a amostragem não converte orçamento em vantagem. Em uma fração substancial das tarefas, todos os candidatos amostrados empatam em zero pares de demonstração corretos; com o desempate pela primeira amostra, o *best-of-N* torna-se operacionalmente idêntico a uma tentativa única. Nas tarefas difíceis do ARC, o sinal de treino é fraco demais para ordenar candidatos. Esse é precisamente o regime em que um crítico com informação externa deveria oferecer maior ganho, e onde uma intervenção futura tem mais a explorar do que no regime médio capturado pela acurácia agregada.

5.3 Limitações e Ameaças à Validade

1. **Metade do resultado vem de uma chamada comum às condições.** É a limitação dominante (Seção 4.2). A correção é conhecida e barata — um *cache* de semente — e não foi aplicada aqui.
2. **Confundimento temporal na linha de base.** *sampling* foi coletado em 21/08/2026 e as condições de crítico em 02–03/09/2026. O modelo é hospedado e pode ter mudado do lado do servidor no intervalo; nenhum *digest* de *prompt* captura isso. As comparações 1, 2 e 3 carregam esse confundimento; as comparações 4 e 5, entre condições de crítico coletadas juntas, não.
3. **Confundimento múltiplo entre os braços.** *sampling* e as condições de crítico diferem em quatro eixos simultaneamente. A decomposição fatorial endereça dois deles, mas um resultado positivo sustentaria a leitura pragmática, não a causal.
4. **Os oráculos não são autônomos.** *critic* e *critic_cegis* consultam o gabarito do teste durante o laço; medem o valor da validação por oposição como limite superior, não um método implantável. Apenas *critic_no_oracle* descreve um sistema executável.
5. **O orçamento é medido em chamadas, não em tokens,** com a consequência já descrita.
6. **Uma execução por tarefa.** Sem repetições, parte da variação é ruído do modelo, ampliado em *sampling* pela temperatura alta. O pareamento mitiga, não elimina — e a instabilidade de casos individuais entre execuções é visível na Seção 4.3.
7. **Análises interinas durante a coleta** inflam o erro tipo I. Os *p*-valores reportados são os da análise pré-registrada sobre o conjunto fechado; a Seção 4.3 quantifica o custo de tê-las consultado.
8. **Vazamento residual.** O filtro captura grades e código, não paráfrases. A contagem de 0 redações mede o que foi barrado, não o que passou.
9. **Dependência de modelo.** A acurácia absoluta e a própria existência de pares discordantes dependem do modelo (Seção 3.6). As conclusões referem-se ao regime de capacidade de *gemini-3.5-flash-lite*.

6 Conclusão

Sob orçamento fixo de chamadas a um modelo de linguagem para síntese de programas no ARC-AGI-1, quatro condições

que variam isoladamente o acesso ao oráculo e a forma do feedback não se separam em 270 tarefas: **nenhuma** das 5 comparações pré-registradas é significativa — nem sob Bonferroni, nem a $\alpha = 0,05$ sem correção. O crítico opera como mecanismo, atingindo a mesma consistência com os pares de demonstração com 40% menos programas, sem que isso se converta em acerto no par de teste; e uma medição independente sobre as mesmas tarefas confirma que a revisão guiada por oráculo **não** superou a amostragem independente.

O resultado mais transferível é outro, e independe de qual condição venha a prevalecer: cerca de metade das vitórias de qualquer condição é decidida na primeira geração, que é idêntica entre elas. Descontada essa parcela, a medição discrimina em cerca de um quinto da amostra, e a ordenação entre as condições inverte em relação ao número bruto. Qualquer comparação entre esquemas de inferência que partam do mesmo *prompt* inicial herda esse confundimento, e nenhuma métrica agregada o revela.

Três contribuições permanecem. O protocolo: orçamento igualado, desenho pareado, oráculo que informa mas não seleciona, filtro de vazamento auditável — que registrou 0 eventos em 1.844 críticas — e família de comparações pré-registrada. O desconto da primeira geração, com a correção correspondente identificada. E a medida de quanto uma ordenação entre condições é reproduzível: em 60 tarefas, a ordenação completa das quatro se repete em 12% das subamostras, e extrapolar uma análise interina projetou $p = 0,0027$ para um resultado que fechou em $p = 0,0789$.

O passo seguinte não é aumentar n . É integrar o *cache* de semente, que elimina o confundimento dominante e reduz o custo de um bloco de quatro condições em 810 chamadas; medir o orçamento em tokens, que é a unidade de custo efetiva e pode inverter o sinal da comparação; e concentrar a medição no regime em que a amostragem comprovadamente não converte orçamento em vantagem — as tarefas em que todos os candidatos empatam em zero pares de demonstração corretos —, onde um crítico informado tem, em princípio, mais a oferecer.

Reprodutibilidade

O ambiente é gerenciado por `uv`, com interpretador fixado e versões de dependências travadas em arquivo de *lock* versionado. Cada rodada grava um manifesto com a lista literal de identificadores de tarefa, a semente, o orçamento, os modelos e o número (nunca a identidade) de chaves de API utilizadas, além de um registro `jsonl` por condição contendo todos os *prompts*, respostas brutas, contagens de token, códigos gerados e resultados de execução por tarefa. As respostas brutas do Crítico são preservadas antes da aplicação do filtro antivazamento, o que permite auditar independentemente tanto o que foi redigido quanto o que passou. Um modo de execução seca percorre o *pipeline* inteiro sem consumir API, para verificação de instalação. Os testes automatizados travam as propriedades críticas do desenho, incluindo a separação de informação entre os *prompts* do Gerador e do Crítico e a paridade de orçamento entre as condições.

Referências

- Alur, R.; Bodík, R.; Juniwal, G.; Martin, M. M. K.; Raghothaman, M.; Seshia, S. A.; Singh, R.; Solar-Lezama, A.; Torlak, E.; and Udupa, A. 2013. Syntax-Guided Synthesis. In *Formal Methods in Computer-Aided Design (FMCAD)*, 1–8.
- Brown, B.; Juravsky, J.; Ehrlich, R.; Clark, R.; Le, Q. V.; Ré, C.; and Mirhoseini, A. 2024. Large Language Monkeys: Scaling Inference Compute with Repeated Sampling. arXiv:2407.21787.
- Button, K. S.; Ioannidis, J. P. A.; Mokrysz, C.; Nosek, B. A.; Flint, J.; Robinson, E. S. J.; and Munafò, M. R. 2013. Power Failure: Why Small Sample Size Undermines the Reliability of Neuroscience. *Nature Reviews Neuroscience*, 14(5): 365–376.
- Chen, M.; Tworek, J.; Jun, H.; Yuan, Q.; Pinto, H. P. d. O.; Kaplan, J.; Edwards, H.; Burda, Y.; Joseph, N.; Brockman, G.; et al. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374.
- Chen, X.; Lin, M.; Schärli, N.; and Zhou, D. 2024. Teaching Large Language Models to Self-Debug. In *The Twelfth International Conference on Learning Representations (ICLR)*.
- Chollet, F. 2019. On the Measure of Intelligence. arXiv:1911.01547.
- Chollet, F.; Knoop, M.; Kamradt, G.; and Landers, B. 2024. ARC Prize 2024: Technical Report. arXiv:2412.04604.
- Cobbe, K.; Kosaraju, V.; Bavarian, M.; Chen, M.; Jun, H.; Kaiser, L.; Plappert, M.; Tworek, J.; Hilton, J.; Nakano, R.; Hesse, C.; and Schulman, J. 2021. Training Verifiers to Solve Math Word Problems. arXiv:2110.14168.
- Dunn, O. J. 1961. Multiple Comparisons Among Means. *Journal of the American Statistical Association*, 56(293): 52–64.
- Ellis, K.; Wong, C.; Nye, M.; Sablé-Meyer, M.; Morales, L.; Hewitt, L.; Cary, L.; Solar-Lezama, A.; and Tenenbaum, J. B. 2021. DreamCoder: Bootstrapping Inductive Program Synthesis with Wake-Sleep Library Learning. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI)*, 835–850.
- Gelman, A.; and Loken, E. 2014. The Statistical Crisis in Science. *American Scientist*, 102(6): 460–465.
- Gendron, G.; Bao, Q.; Witbrock, M.; and Dobbie, G. 2024. Large Language Models Are Not Strong Abstract Reasoners. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI-24)*, 6270–6278.
- Greenblatt, R. 2024. Getting 50% (SoTA) on ARC-AGI with GPT-4o. <https://redwoodresearch.substack.com/p/getting-50-sota-on-arc-agi-with-gpt>. Accessed: 2026-08-28.
- Gulwani, S.; Polozov, O.; and Singh, R. 2017. Program Synthesis. *Foundations and Trends in Programming Languages*, 4(1-2): 1–119.
- Huang, J.; Chen, X.; Mishra, S.; Zheng, H. S.; Yu, A. W.; Song, X.; and Zhou, D. 2024. Large Language Models Cannot Self-Correct Reasoning Yet. In *The Twelfth International Conference on Learning Representations (ICLR)*.

Ioannidis, J. P. A. 2005. Why Most Published Research Findings Are False. *PLoS Medicine*, 2(8): e124.

Li, Y.; Choi, D.; Chung, J.; Kushman, N.; Schrittwieser, J.; Leblond, R.; Eccles, T.; Keeling, J.; Gimeno, F.; Dal Lago, A.; et al. 2022. Competition-Level Code Generation with AlphaCode. *Science*, 378(6624): 1092–1097.

Lightman, H.; Kosaraju, V.; Burda, Y.; Edwards, H.; Baker, B.; Lee, T.; Leike, J.; Schulman, J.; Sutskever, I.; and Cobbe, K. 2024. Let’s Verify Step by Step. In *The Twelfth International Conference on Learning Representations (ICLR)*.

Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegrefe, S.; Alon, U.; Dziri, N.; Prabhunoye, S.; Yang, Y.; et al. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 46534–46594.

McNemar, Q. 1947. Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages. *Psychometrika*, 12(2): 153–157.

Mirchandani, S.; Xia, F.; Florence, P.; Ichter, B.; Driess, D.; Arenas, M. G.; Rao, K.; Sadigh, D.; and Zeng, A. 2023. Large Language Models as General Pattern Machines. In *Proceedings of the 7th Conference on Robot Learning (CoRL)*, 2498–2518.

Olausson, T. X.; Inala, J. P.; Tan, C.; Gao, J.; and Solar-Lezama, A. 2024. Is Self-Repair a Silver Bullet for Code Generation? In *The Twelfth International Conference on Learning Representations (ICLR)*.

Pocock, S. J. 1977. Group Sequential Methods in the Design and Analysis of Clinical Trials. *Biometrika*, 64(2): 191–199.

Shinn, N.; Cassano, F.; Berman, E.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 8634–8652.

Snell, C.; Lee, J.; Xu, K.; and Kumar, A. 2024. Scaling LLM Test-Time Compute Optimally Can Be More Effective Than Scaling Model Parameters. arXiv:2408.03314.

Solar-Lezama, A.; Tancau, L.; Bodik, R.; Seshia, S.; and Saraswat, V. 2006. Combinatorial Sketching for Finite Programs. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 404–415.

Stechly, K.; Marquez, M.; and Kambhampati, S. 2023. GPT-4 Doesn’t Know It’s Wrong: An Analysis of Iterative Prompting for Reasoning Problems. arXiv:2310.12397.

Wang, R.; Zelikman, E.; Poesia, G.; Pu, Y.; Haber, N.; and Goodman, N. D. 2024. Hypothesis Search: Inductive Reasoning with Language Models. In *The Twelfth International Conference on Learning Representations (ICLR)*.

Wang, X.; Wei, J.; Schuurmans, D.; Le, Q. V.; Chi, E. H.; Narang, S.; Chowdhery, A.; and Zhou, D. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *The Eleventh International Conference on Learning Representations (ICLR)*.

Wilson, E. B. 1927. Probable Inference, the Law of Succession, and Statistical Inference. *Journal of the American Statistical Association*, 22(158): 209–212.

Apêndice A: Execução e Histórico

A.1 A medição sob o protocolo P1

A medição anterior (21/08/2026, `commit 5fdda8d`), descrita na Seção 3.8, avaliou apenas `sampling` e `critic` sobre as mesmas 270 tarefas, sob o protocolo P1. A Seção 4.2 a usa como réplica independente do desconto da primeira geração; os agregados completos estão na Tabela 7.

Condição	Res.	Acur.	Treino	Prog.	Tokens
<code>sampling</code>	88/270	32,6%	35,6%	5,43	7,6 M
<code>critic</code>	86/270	31,9%	35,6%	3,30	12,6 M

Tabela 7: Agregados da medição sob P1.

A tabela de contingência pareada divide-se em 62 tarefas resolvidas pelas duas condições, 158 por nenhuma, e 50 discordâncias (18,5% das tarefas) repartidas 24×26 — a base do $p = 0,8877$ (Equação 2) e do intervalo de $-5,6$ a $+4,3$ pp (Equação 3) reportados na Seção 4.2.

A explicação candidata para o resultado nulo — que o Crítico induziria maior sobreajuste aos pares de demonstração — foi registrada como exploratória e testada nas tarefas novas isoladamente, sem se replicar (23,3% na análise interina contra 15,2% na réplica, $p = 0,6173$).

A.2 Os dois protocolos do Crítico

Sob **P1**, a mensagem enviada ao Crítico não continha o código candidato, apenas a regra e o relatório de execução. Uma crítica típica opera, por isso, sobre a descrição do algoritmo:

“The stated rule and code incorrectly assume an intersection-based logical operation. . . converting many incorrect cells. . .”

Sob **P2**, com o código incluído, o Crítico passa a comparar a implementação com a regra enunciada — comportamento estruturalmente impossível antes:

“## CODE/RULE MISMATCH The code implements line drawing and overwriting order based on iteration sequence, but it completely fails to implement the precedence or layering rule described in item 3 of the stated rule. Instead, whichever line type (row or column) is processed last simply overwrites the intersecting cells.”

Esta é a crítica citada na Seção 4.4: emitida por `critic_no_oracle`, que nunca vê o gabarito, ela localiza o defeito por consistência interna entre código e regra.

A.3 O cache de semente

A correção para o confundimento da primeira geração (Seção 4.2) está implementada no `commit 19e492c` do repositório, em ramo não integrado. Nenhuma das medições reportadas neste trabalho o utiliza.

A.4 Notas de execução

A medição sob P1 consumiu 1.812 chamadas em 20 minutos com 7 chaves de API em paralelo. A medição principal consumiu 4.601 chamadas ao longo de dois dias, cobrindo

810 execuções de condição-tarefa e 1.844 críticas emitidas, com 11 chaves úteis; registrou 0 erros de API, 13 respostas truncadas e 0 eventos de vazamento. A cota efetivamente medida foi de ~ 230 chamadas por chave por dia, contra as 500 anunciadas — informação relevante para dimensionar uma réplica.

A.5 Execuções-piloto e a banda predita

Antes da medição principal, a mesma configuração foi executada em 30 e em 60 tarefas. Essas execuções não entram na análise: a de 60 tarefas foi descartada e refeita do zero ao se estender a amostra, e ambas medem execuções distintas, não subconjuntos da medição final. Servem apenas como verificação externa da Figura 5.

A diferença `critic menos sampling` observada foi de $-13,4$ pp na execução de 30 tarefas e de $+6,6$ pp na de 60. A banda de 5 a 95% prevista por subamostragem é de $-10,0$ a $+13,3$ pp em 30 tarefas e de $-6,7$ a $+10,0$ pp em 60. O valor de 60 tarefas cai dentro da banda; o de 30 cai fora dela. A subamostragem mede apenas variância de amostra, enquanto uma reexecução acrescenta a variância do próprio modelo — o excesso observado é consistente com isso, e reforça que os valores da Seção 4.3 são um piso otimista.